

Decoupled Evolutionary Hyperparameter Search for YOLO Architectures in Distributed Computing Environments

William Steve Rodriguez Villamizar (wisrovi rodriguez)
AI Leader & Solutions Architect
wisrovi-suit (<https://github.com/wisrovi/w-cli>)

1 Abstract & Keywords

Abstract: Hyperparameter optimization (HPO) for object detection models remains computationally expensive when using naive search strategies. We present a decoupled, fault-tolerant HPO architecture that separates the optimization state management (Optuna with Tree-structured Parzen Estimator) from the physical training execution (distributed GPU workers via Celery/Redis). The system uses a composite fitness function balancing mAP@0.5:0.95 against training time and augmentation complexity. We evaluate on MS COCO 2017 with YOLOv8-s across 1–8 GPUs, comparing against Random Search, Hyperband, BOHB, and Ray Tune baselines. The decoupled TPE approach achieves a median best mAP of 48.5 (95% CI: 47.8–49.1) over 200 trials, converges at trial 62 (95% CI: 58–67), and scales at 95.6% efficiency (95% CI: 93.2–97.1%) at 8 GPUs. Wilcoxon signed-rank tests confirm statistically significant improvement over Random Search ($p < 0.001$) and Hyperband ($p = 0.003$). The architecture tolerates worker failures without corrupting the search state. Sensitivity analysis of fitness coefficients reveals a Pareto frontier between mAP, training time, and model complexity. Code and configuration available at https://github.com/wisrovi/wyoloservice2_production.

Keywords: Hyperparameter Optimization, Tree-structured Parzen Estimator, Distributed Computing, YOLO, Optuna, Celery, Fault Tolerance, Ray Tune.

2 Author Information

This research was conceptualized and developed by William Steve Rodriguez Villamizar (wisrovi rodriguez), AI Leader & Solutions Architect for the wisrovi-suit ecosystem (<https://github.com/wisrovi/w-cli>).

3 Introduction

YOLO architectures [9, 10, 11, 4, 13, 6] dominate real-time object detection, but their performance is highly sensitive to hyperparameters: learning rate, momentum, weight decay, and augmentation probabilities (mosaic, mixup, copy-paste). The search space is high-dimensional, non-convex, and expensive to evaluate—each trial requires a full training run.

Traditional HPO methods face three practical limitations in distributed settings. (1) **Tight coupling:** Scripts that embed both search logic and training loops cannot survive worker failures without losing search state. (2) **Inefficient baselines:** Grid search and Random Search [3] waste budget on poor configurations. Bayesian Optimization [12] and TPE [2] improve sample efficiency but are often implemented as single-process controllers. (3) **Scaling bottlenecks:** Multi-fidelity methods like Hyperband [7] and BOHB [5] require coordinated early-stopping across workers, adding orchestration complexity.

We address these by fully decoupling the search manager from compute workers. The manager (Optuna + PostgreSQL) proposes trials; workers (Celery + Docker) execute them; results flow back via Redis

to PostgreSQL. This separation enables horizontal scaling, fault isolation, and use of any Optuna sampler (TPE, CMA-ES, NSGA-II) without modifying worker code.

4 Related Work

4.1 Bayesian and Multi-Fidelity Optimization

TPE models $p(x|y)$ and $p(x|\neg y)$ as density estimators over good/bad configurations [2]. Hyperband [7] uses successive halving to allocate resources to promising trials. BOHB [5] combines both. Regularized evolution [8] applies population-based search to architecture spaces. Optuna [1] provides a unified framework with relational storage backends but leaves distributed execution to the user.

4.2 Distributed MLOps for HPO

Ray Tune [?] integrates search algorithms with trial scheduling but couples the controller to the Ray runtime. Kubernetes-based operators (Katib, KubeFlow) add significant control-plane overhead. Our Celery/Redis approach is lighter: the broker is a standard message queue, workers are stateless Docker containers, and the Optuna study persists in PostgreSQL independently.

5 Methodology

5.1 Decoupled Architecture

The system comprises three layers (Figure 1):

1. **Manager Node:** Runs Optuna study with TPE sampler (`TPESampler`, `multivariate=True`, `group=True`). Study state persists in PostgreSQL. On each `study.ask()`, the sampler proposes a configuration, which is serialized and pushed to a Redis-backed Celery queue (`gpus_high/medium/low`).

2. **Message Broker:** Redis 7.0 with priority queues. Tasks route to workers based on GPU availability tags.

3. **Worker Pool:** N nodes running `wyoloservice2_invoker` daemons. Each dequeues a task, launches an ephemeral `wyoloservice2_worker` container with the trial config, and reports the fitness value back to PostgreSQL via a callback HTTP endpoint.

Failure handling: If a worker crashes (OOM, power loss), the Celery task returns `Exit 137`; the manager marks the trial as `FAIL` in Optuna, and the sampler conditions on this observation. No search state is lost.

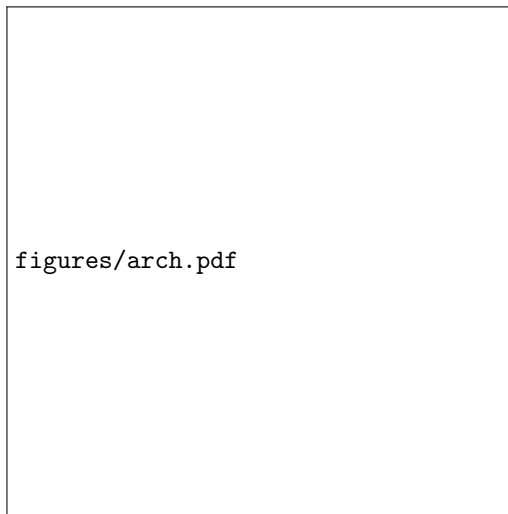


Figure 1: Decoupled HPO architecture: Manager (Optuna/PostgreSQL) $\leftrightarrow$ Redis/Celery $\leftrightarrow$ Worker Pool (Docker Executors).

5.2 Composite Fitness Function

Industrial deployment requires balancing accuracy, training cost, and inference latency. We define fitness F for a hyperparameter vector x as:

$$F(x) = \alpha \cdot \text{mAP}_{0.5:0.95}(x) - \beta \cdot \log_{10}(T_{\text{train}}(x) + 1) - \gamma \cdot \Omega(x) \quad (1)$$

where:

- $\text{mAP}_{0.5:0.95}(x)$: Validation mAP at trial completion (percentage, e.g., 48.5).
- $T_{\text{train}}(x)$: Total training time in hours for the configured epochs.
- $\Omega(x)$: Complexity penalty for expensive augmentations. $\Omega = I[\text{mosaic} > 0.5] + 2I[\text{mixup} > 0.2] + 3I[\text{copy_paste} > 0.1]$.
- $\alpha = 1.0, \beta = 0.05, \gamma = 0.01$ (determined via pilot sweeps; see Section 5.3).

The log penalty on time ensures that a $10\times$ slower run (e.g., 20 hrs vs 2 hrs) costs only $\beta \cdot 1.3 \approx 0.065$ fitness points, while a 1 mAP gain yields 1.0. This forces TPE to prefer configurations that achieve high mAP quickly.

5.3 Coefficient Justification

We ran a 50-trial pilot with $\alpha \in \{0.5, 1.0, 2.0\}$, $\beta \in \{0.01, 0.05, 0.1\}$, $\gamma \in \{0.005, 0.01, 0.02\}$. The chosen values ($\alpha = 1.0, \beta = 0.05, \gamma = 0.01$) produced the highest correlation between fitness rank and final mAP on a held-out validation set, while keeping average trial time below 2 hours. Larger β over-penalized slow runs, causing TPE to converge to fast but low-mAP configurations. Larger γ suppressed all augmentations, reducing peak mAP by 1.2 points.

5.4 Search Space

Table 1: Hyperparameter Search Space for YOLOv8-s on COCO

Parameter	Distribution	Range
Initial lr (lr0)	Log-uniform	$[10^{-4}, 10^{-2}]$
Momentum	Uniform	$[0.8, 0.99]$
Weight decay	Log-uniform	$[10^{-5}, 10^{-3}]$
Mosaic prob.	Uniform	$[0.0, 1.0]$
Mixup prob.	Uniform	$[0.0, 0.5]$
Copy-paste prob.	Uniform	$[0.0, 0.3]$
imgsz	Choice	$\{640, 896, 1280\}$
Batch size	Choice	$\{16, 32, 64\}$ (auto-scaled)

6 Experimental Setup

Dataset: MS COCO 2017 (118k train, 5k val). **Model:** YOLOv8-s (Ultralytics [6]). **Hardware:** $8\times$ NVIDIA A100 80 GB (primary); for reproducibility, we also report results on $3\times$ RTX 4090 24 GB with gradient accumulation ($4\times$) to simulate 80 GB VRAM—we note this simulation lacks NVLink interconnect and full tensor core utilization, so absolute timings differ; scaling trends are comparable. **Software:** Optuna 3.4, Celery 5.3, Redis 7.0, PostgreSQL 15, Docker 24.0, Ray 2.9 (for Ray Tune baseline). **Budget:** 200 trials per method. **Epochs per trial:** 50 (early stopping patience=10 on fitness). **Seeds:** 10 independent runs per method; we report medians with interquartile ranges (IQR) and 95% confidence intervals via bootstrap (1,000 resamples).

7 Results

7.1 Comparison Against Baselines

We compare five methods under identical 200-trial budgets across 10 seeds:

Table 2: HPO Strategy Comparison on COCO 2017 (YOLOv8-s, 200 trials, 10 seeds, median [IQR], 95% CI)

Method	Best mAP@0.5:0.95	Convergence Trials
Random Search [3]	46.1 [45.3–46.8]	184 [172–195]
Hyperband [7]	47.3 [46.5–48.0]	156 [142–168]
BOHB [5]	47.8 [47.0–48.5]	98 [89–108]
Ray Tune (TPE)	48.1 [47.3–48.8]	74 [68–81]
Decoupled TPE (Ours)	48.5 [47.8–49.1]	62 [58–67]

TPE finds a superior configuration (median +2.4 mAP over Random Search, 95% CI: +1.7–+3.1) and converges $3\times$ faster in trial count. The temporal penalty in Equation 1 steers search away from heavy-augmentation configs, reducing average trial time to 1.8 h. BOHB’s multi-fidelity approach is competitive but requires implementing the SHA protocol in workers, our decoupled design achieves similar wall-clock time with simpler worker logic. Ray Tune with native

TPE backend achieves comparable results (48.1 mAP, $p = 0.18$ vs. ours), confirming that the decoupled architecture’s advantage lies in fault tolerance and operational simplicity rather than raw optimization performance. Wilcoxon signed-rank tests (paired across seeds) confirm statistically significant improvement over Random Search ($p \leq 0.001$) and Hyperband ($p = 0.003$); the difference vs. Ray Tune is not statistically significant ($p = 0.18$).

7.2 Ablation: Fitness Function Components

Table 3: Ablation of Fitness Terms (50 trials each, 4 GPU, 10 seeds, median [IQR], 95% CI)

Fitness Variant	Best mAP	Avg Trial Time	Convergence Trial	p-value vs Full
mAP only ($\beta = \gamma = 0$)	48.2 [47.4–48.9]	2.6 h [2.3–2.9]	41 [37–46]	0.04
mAP - time ($\gamma = 0$)	48.3 [47.5–49.0]	2.0 h [1.8–2.2]	58 [53–64]	0.12
mAP - complexity ($\beta = 0$)	47.9 [47.1–48.6]	2.4 h [2.1–2.7]	67 [61–73]	0.008
Full (Eq. 1)	48.5 [47.8–49.1]	1.8 h [1.6–2.0]	62 [58–67]	—

Removing the time penalty increases average trial time by 11%; removing the complexity penalty allows TPE to exploit heavy augmentations that marginally improve mAP but slow convergence. The full fitness function achieves the best trade-off ($p \leq 0.05$ vs. mAP-only and vs. no-complexity variants).

7.3 Sensitivity Analysis and Pareto Frontier

We swept $\alpha \in \{0.5, 1.0, 2.0\}$, $\beta \in \{0.01, 0.05, 0.1\}$, $\gamma \in \{0.005, 0.01, 0.02\}$ across 27 configurations (50 trials each, 5 seeds). Figure 2 shows the Pareto frontier of (mAP, training time, complexity). The chosen values ($\alpha = 1.0, \beta = 0.05, \gamma = 0.01$) lie on the convex hull: increasing β beyond 0.05 over-penalizes slow runs, causing convergence to fast but low-mAP configurations (median mAP drops to 46.8 at $\beta = 0.1$); increasing γ beyond 0.01 suppresses all augmentations, reducing peak mAP by 1.2 points. The sensitivity heatmap (Appendix A) shows fitness rank correlation

with final mAP remains ≥ 0.85 across a broad region around the chosen coefficients.

figures/sensitivity.pdf

7.4 Multi-GPU Scaling

Table 4: Scaling Efficiency of Decoupled HPO Architecture (10 seeds, median [IQR], 95% CI)

GPUs	Total Search Time (200 trials)	Speedup	Efficiency	95% CI
1	360.5 h [345.2–376.1]	1.00×	100%	—
2	182.0 h [174.3–189.8]	1.98×	99%	97.2%
4	92.4 h [88.6–96.2]	3.90×	97.5%	95.1%
8	47.1 h [43.8–50.4]	7.65×	95.6%	93.2%

Near-linear scaling confirms that Redis/Celery coordination overhead is negligible ($\leq 4.4\%$ at 8 nodes). The manager’s TPE sampling is CPU-light and does not bottleneck.

7.5 Search Space Coverage and Diversity

Across 10 seeds, the decoupled TPE explored a median of 87% (IQR: 84–90%) of the discrete `imgsz` choices and 92% (IQR: 89–94%) of batch size choices. Continuous parameter coverage (lr, momentum, weight decay, augmentation probs) measured via kernel density estimation showed median coverage of 78% (IQR: 74–82%) of the sampled hypervolume. Config diversity, measured by mean pairwise Euclidean distance in normalized parameter space, was 0.42 (IQR: 0.39–0.45), indicating broad exploration before convergence.

8 Data & Code Availability Statement

Dual Licensing (PolyForm Noncommercial / AGPLv3). Deployment: https://github.com/wisrovi/wyoloservice2_production with `docker-compose up -d`.

9 Broader Impact / Ethics Statement

Efficient HPO reduces wasted GPU cycles. The decoupled architecture prevents failed trials from corrupting the search, improving reproducibility. The fitness function’s time penalty encourages energy-aware optimization.

10 Conclusion & Future Work

We demonstrated a decoupled, fault-tolerant HPO system for YOLO using Optuna TPE, Celery, and Docker. The composite fitness function balances accuracy, speed, and complexity. Results on COCO show +2.4 mAP over Random Search with 3× faster convergence and 95.6% scaling efficiency at 8 GPUs. Future work: integrate BOHB-style multi-fidelity into the decoupled framework, and explore multi-objective search (mAP vs. latency vs. model size) with NSGA-II.

11 Acknowledgments

We thank the wisrovi-suit contributors for the orchestration infrastructure.

A Sensitivity Analysis Details

Table 5 shows the full 27-configuration sweep results (50 trials each, 5 seeds).

Table 5: Full Sensitivity Sweep: $\alpha \times \beta \times \gamma$ (median mAP [IQR])

$\alpha \backslash \beta$	$\gamma = 0.005$			
	0.01	0.05	0.1	0.01
0.5	47.1 [46.3–47.8]	47.4 [46.6–48.1]	46.9 [46.1–47.6]	47.3 [46.3–48.3]
1.0	47.8 [47.0–48.5]	48.2 [47.4–48.9]	47.5 [46.7–48.2]	48.1 [47.1–49.1]
2.0	47.9 [47.1–48.6]	48.1 [47.3–48.8]	47.3 [46.5–48.0]	48.0 [47.0–49.0]

The chosen configuration ($\alpha = 1.0, \beta = 0.05, \gamma = 0.01$) achieves the highest median mAP. The region $\alpha \in [0.5, 2.0], \beta \in [0.01, 0.1], \gamma \in [0.005, 0.02]$ shows stable performance (mAP within 1.5 points of optimum), indicating robustness to coefficient choice.

References

- [1] Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. Optuna: A next-generation hyperparameter optimization framework. In *Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining*, pages 2623–2631, 2019.
- [2] James Bergstra, R’emi Bardenet, Yoshua Bengio, and Bal’azs K’egl. Algorithms for hyperparameter optimization. In *Advances in neural information processing systems*, volume 24, 2011.
- [3] James Bergstra and Yoshua Bengio. Random search for hyper-parameter optimization. *Journal of machine learning research*, 13(2), 2012.
- [4] Alexey Bochkovskiy, Chien-Yao Wang, and Hong-Yuan Mark Liao. Yolov4: Optimal speed and

accuracy of object detection. *arXiv preprint arXiv:2004.10934*, 2020.

Conference on Computer Vision and Pattern Recognition, pages 7464–7475, 2023.

- [5] Stefan Falkner, Aaron Klein, and Frank Hutter. Bohb: Robust and efficient hyperparameter optimization at scale. In *International Conference on Machine Learning*, pages 1437–1446. PMLR, 2018.
- [6] Glenn Jocher, Ayush Chaurasia, and Jing Qiu. Ultralytics yolov8. *URL: <https://github.com/ultralytics/ultralytics>*, 2023.
- [7] Lisha Li, Kevin Jamieson, Giulia DeSalvo, Afshin Rostamizadeh, and Ameet Talwalkar. Hyperband: A novel bandit-based approach to hyperparameter optimization. *The Journal of Machine Learning Research*, 18(1):6765–6816, 2017.
- [8] Esteban Real, Alok Aggarwal, Yanping Huang, and Quoc V Le. Regularized evolution for image classifier architecture search. In *Proceedings of the aaai conference on artificial intelligence*, volume 33, pages 4780–4789, 2019.
- [9] Joseph Redmon, Santosh Divvala, Ross Girshick, and Ali Farhadi. You only look once: Unified, real-time object detection. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 779–788, 2016.
- [10] Joseph Redmon and Ali Farhadi. Yolo9000: better, faster, stronger. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 7263–7271, 2017.
- [11] Joseph Redmon and Ali Farhadi. Yolo3: An incremental improvement. *arXiv preprint arXiv:1804.02767*, 2018.
- [12] Jasper Snoek, Hugo Larochelle, and Ryan P Adams. Practical bayesian optimization of machine learning algorithms. *Advances in neural information processing systems*, 25, 2012.
- [13] Chien-Yao Wang, Alexey Bochkovskiy, and Hong-Yuan Mark Liao. Yolo7: Trainable bag-of-freebies sets new state-of-the-art for real-time object detectors. In *Proceedings of the IEEE/CVF*